Transfer Learning (or Pretrain and Fine-Tune)

NTV 2026 / HC6a

Today's lecturer: Wilker Aziz

w.aziz@uva.nl

Outline

This class is about **transfer learning** and **pretraining**. That is, the ability to learn reusable model components from datasets covering a variety of more or less closely-related tasks.

Suggested reading:

- For masked language modelling, see Chapter 10 of textbook
- For Transformers, see Chapter 8 of textbook
- For a detailed account of word embeddings, including a historical perspective, see Chapter 5 of textbook.

Many figures are taken or adapted from Lena Voita's NLP Course for You.

Goals

ILOs. After this class the student

- recognise pathways to improve statistical efficiency of NN training via transfer learning
- can explain key auxiliary tasks in pretraining of text encoders;
- can adapt pretrained text encoders for various needs;
- recognise limitations, incl. broader implications, of the pretraining then fine-tune approach to NLP.

Table of contents

1. Transfer learning

2. Auxiliary tasks

   Word representation

   Representing words in context

3. BERT-style encoders

4. Wrap-up

Transfer learning

Feature Learning

In week 2, we introduced the idea of a **feature function** as a way to represent highly structured inputs (text) in a numerical space.

- This allowed us to realise the mapping from input to a conditional probability distribution over values of the response variable as a trainable (generalised) linear transformation.

In week 3, we realised that we could design the entire **feature function as a composition of trainable components** (a neural network).

- This unlocked all that followed. From powerful encoders to decoders and encoder-decoder models.

But, should we really expect this to work? After all, we are learning the feature function directly from our *limited* task data.

Let’s tune our expectations

Do you expect to learn powerful, predictive features simply because you have an NN?

Let's tune our expectations

Do you expect to learn powerful, predictive features simply because you have an NN? No!

You should expect to learn such useful features when

1. you have an **NN that is capable, in principle, to represent complex features**;
2. you have access to **data that is good enough and abundant enough** for you to learn the parameters of the NN.

Let's tune our expectations

Do you expect to learn powerful, predictive features simply because you have an NN? No!

You should expect to learn such useful features when

1. you have an **NN that is capable, in principle, to represent complex features**;
2. you have access to **data that is good enough and abundant enough** for you to learn the parameters of the NN.

There are (at least) three ways to react to this:

- Go back to feature engineering based on first (say, linguistic) principles
- Annotate more data
- Find an indirect way to obtain more data ← today

Let's focus on neural text classifiers

What do these tasks have in common?

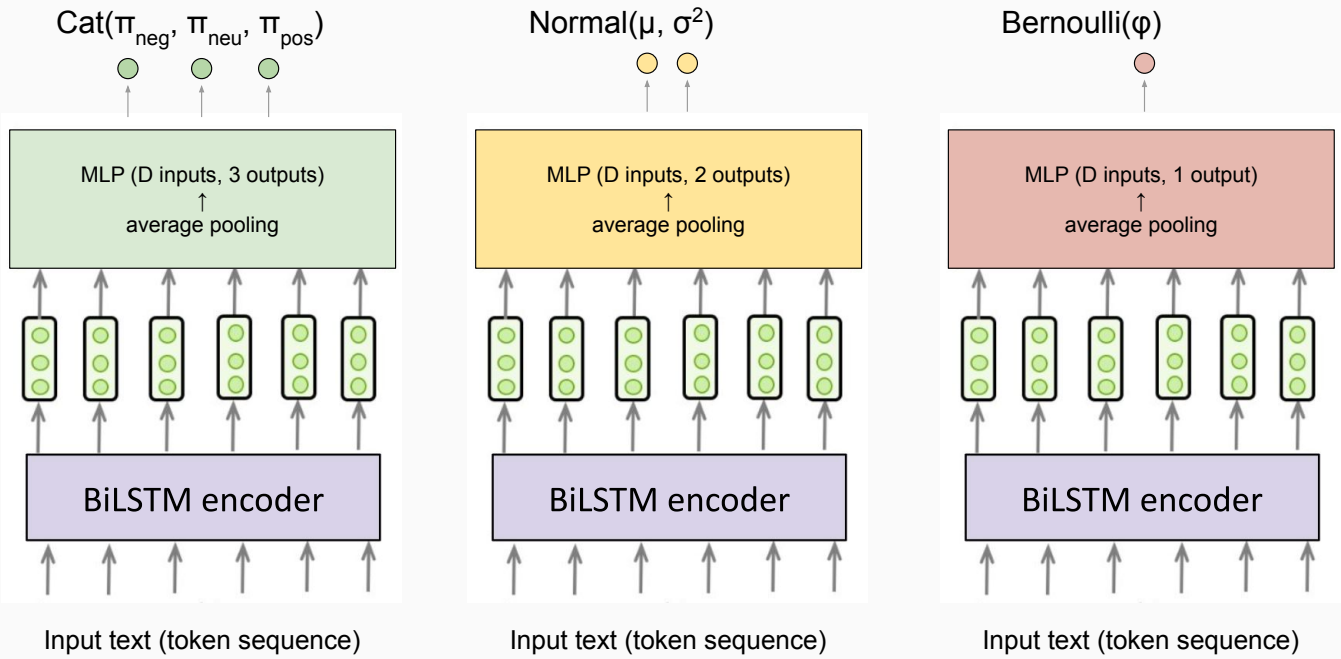
- Predict sentiment about a topic based on a blogpost.
- Predict voting intention for a political party based on threads in a social media platform.
- Predict the readability of a passage of text.

Let's focus on neural text classifiers

What do these tasks have in common?

- Predict sentiment about a topic based on a blogpost.
- Predict voting intention for a political party based on threads in a social media platform.
- Predict the readability of a passage of text.

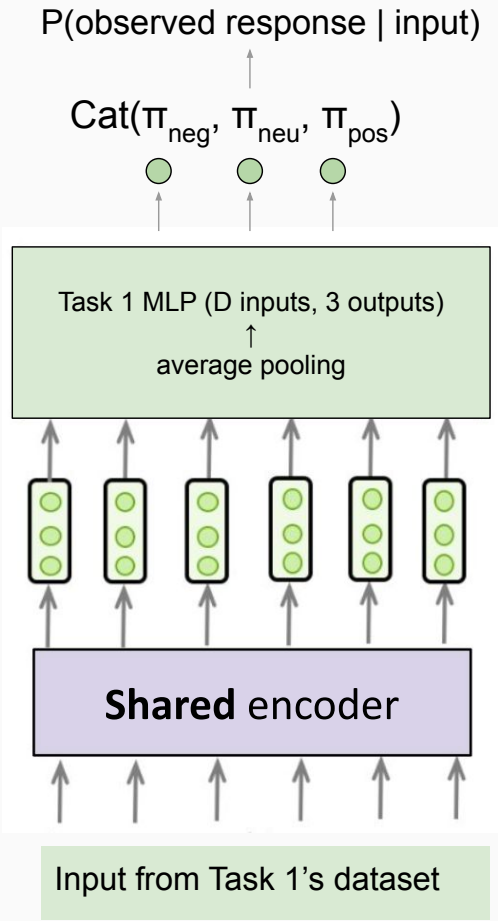
What do these models have in common?



Training under parameter sharing

Here's an idea: what if we shared the encoder across tasks?

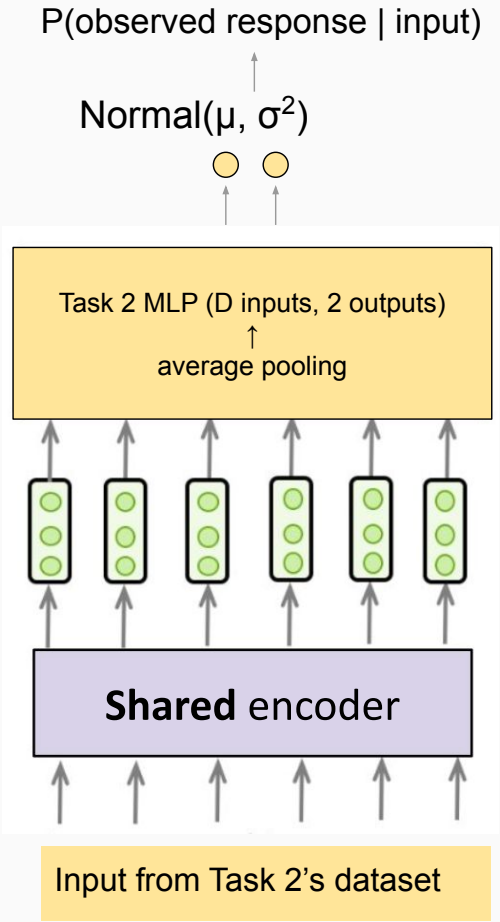
When we observe (x, y) from Task 1, we use, for the the loss definition, the (negative of the **log**) probability that **model 1** assigns to it :



Training under parameter sharing

Here's an idea: what if we shared the encoder across tasks?

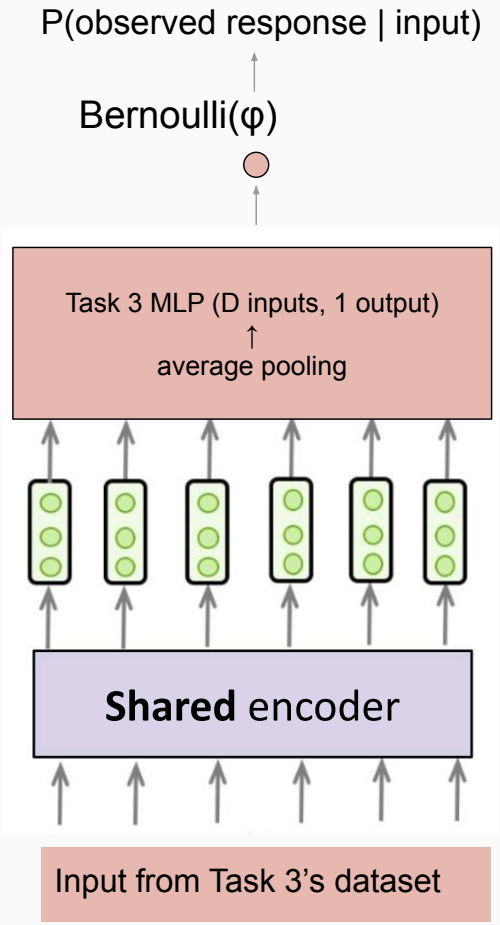
But when we observe (x, y) from Task 2, we use instead the (negative of the log) probability that **model 2** assigns to it:



Training under parameter sharing

Here's an idea: what if we shared the encoder across tasks?

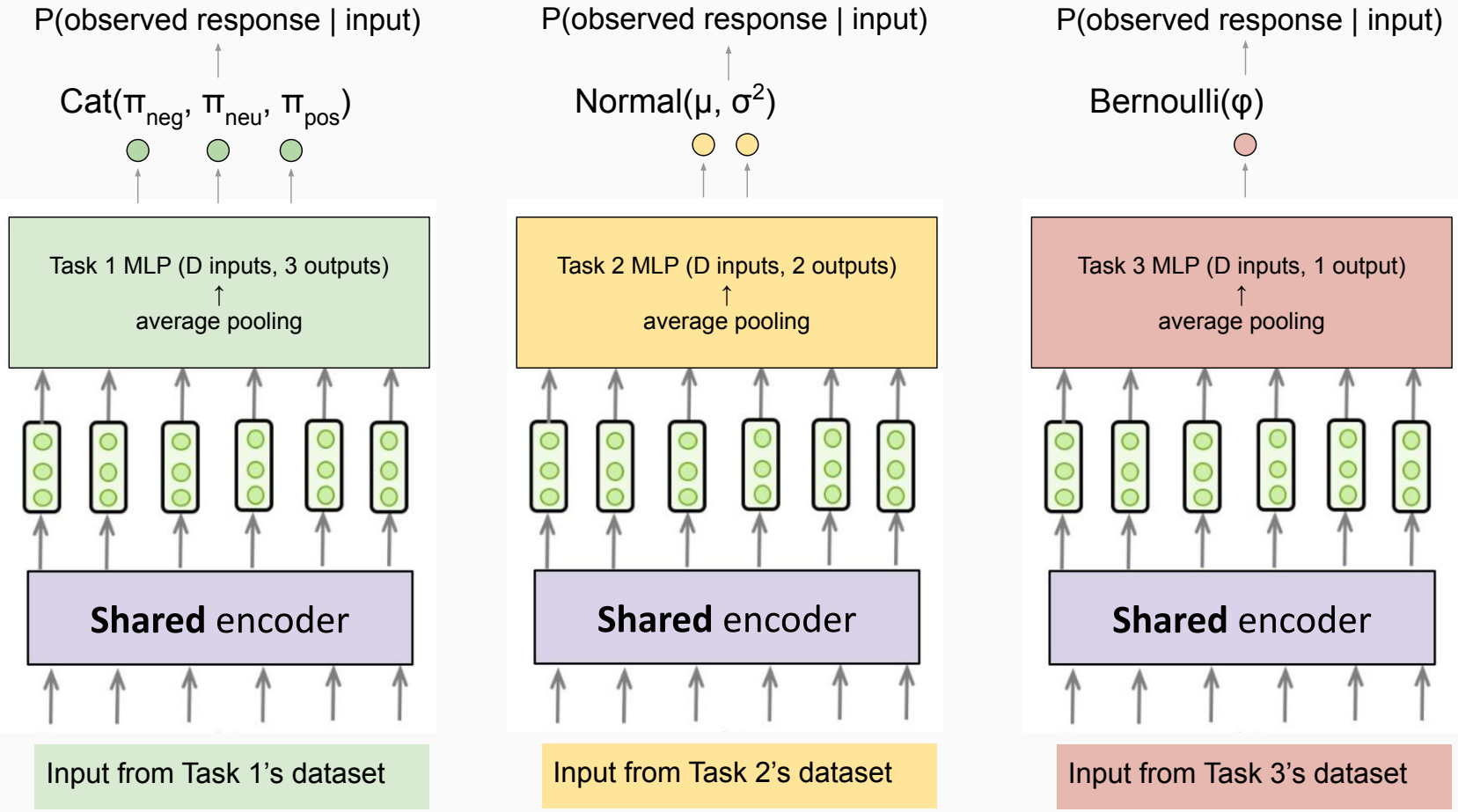
And when we observe (x, y) from Task 3, we use the (negative of the log) probability that **model 3** assigns to it:



Training under parameter sharing

Here's an idea: what if we shared the encoder across tasks?

All models use the same encoder, whose parameters learn from task-specific losses measured in different data points:



Auxiliary tasks

What do models learn?

We like to think that the models learn something about ‘reading English’ simply because **we** see the need to read English as common to all those tasks.

But, **models learn via their predictions not via their inputs**. They learn whatever can improve the loss (reduce mismatch between observed data and model predictions) and not whatever we think might be important.

What do models learn?

We like to think that the models learn something about 'reading English' simply because **we** see the need to read English as common to all those tasks.

But, **models learn via their predictions not via their inputs**. They learn whatever can improve the loss (reduce mismatch between observed data and model predictions) and not whatever we think might be important.

Putting differently: we think of 'learn to read English' as the common aspect across those tasks, but models learn through their losses and their losses aren't about learning to read English (even if they might benefit from a model already knowing English), rather, they are about one classification problem or another.

Auxiliary tasks

An idea that turned out more fruitful is the idea of *constructing* tasks that have much more to do with learning reusable representations of the input data (say, English text) than learning one or another real-world problem.

For example, in linguistics there's an old idea referred to as *distributional semantics*, characterised by the idea that **I can tell a word by the company it keeps**.

Early operational models of distributional semantics represented a word, for example, by counting how often it was seen (in some corpus) in the vicinity of every other word in the vocabulary.[2]

[2] See Baroni et al. [1] for an interesting comparison of classic (count-based) and more modern (prediction-based) models.

Word representation

Let's modernise this count-vector idea and frame it more generally: **the embedding of a word t should store features that are predictive of what words have (or have not) been seen in the neighbourhood of t in some corpus**.

Word representation

Let's modernise this count-vector idea and frame it more generally: **the embedding of a word t should store features that are predictive of what words have (or have not) been seen in the neighbourhood of t in some corpus**.

We can now decide on

- what is an embedding?
- how is it used to predict co-occurrence?
- what is a neighbourhood?
- what corpus should we get observations from?

Word representation

Let's modernise this count-vector idea and frame it more generally: **the embedding of a word t should store features that are predictive of what words have (or have not) been seen in the neighbourhood of t in some corpus**.

We can now decide on

- what is an embedding?
  e.g., a vector of D real values
- how is it used to predict co-occurrence?
  e.g., similarity in embedding space
- what is a neighbourhood?
  e.g., adjacency in fluent text
- what corpus should we get observations from?
  e.g., Wikipedia

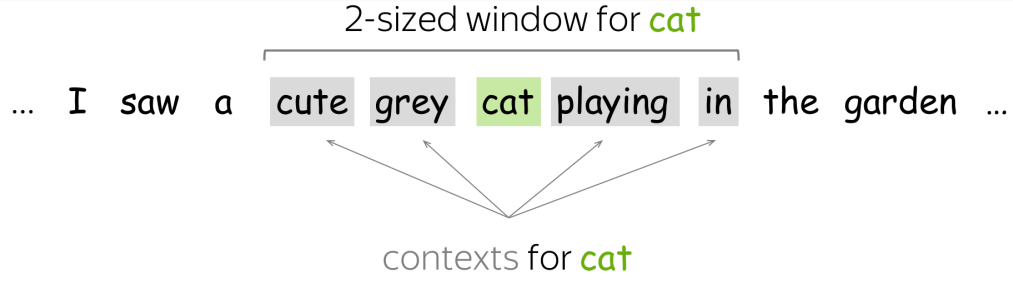
Skip-gram

The model is a simple binary classifier for whether or not two words (t and o) are expected to co-occur $Y = 1$ or not $Y = 0$:

$$Y|W = t, C = o \sim \text{Bernoulli}(\text{sigmoid}(\mathbf{u}^\top \mathbf{v}))$$

$$\text{where } \mathbf{u} = \mathbf{E}_t,\ \mathbf{v} = \mathbf{E}_o \text{ and } \mathbf{E} \in \mathbb{R}^{V \times D}$$

The training data for the skip-gram model consists of pairs of words that **did** co-occur within a window of some predefined size and pairs of words that **did not** co-occur in such a way:



For learning, we can use *any* corpus of English text we like. **There's no real-world task that this data must be designed for**.

We have no use for the binary classifier itself after training, but its embedding matrix $\mathbf{E}$ is reusable across text analysis problems.

What can skip-gram embeddings capture?

Lexical similarity

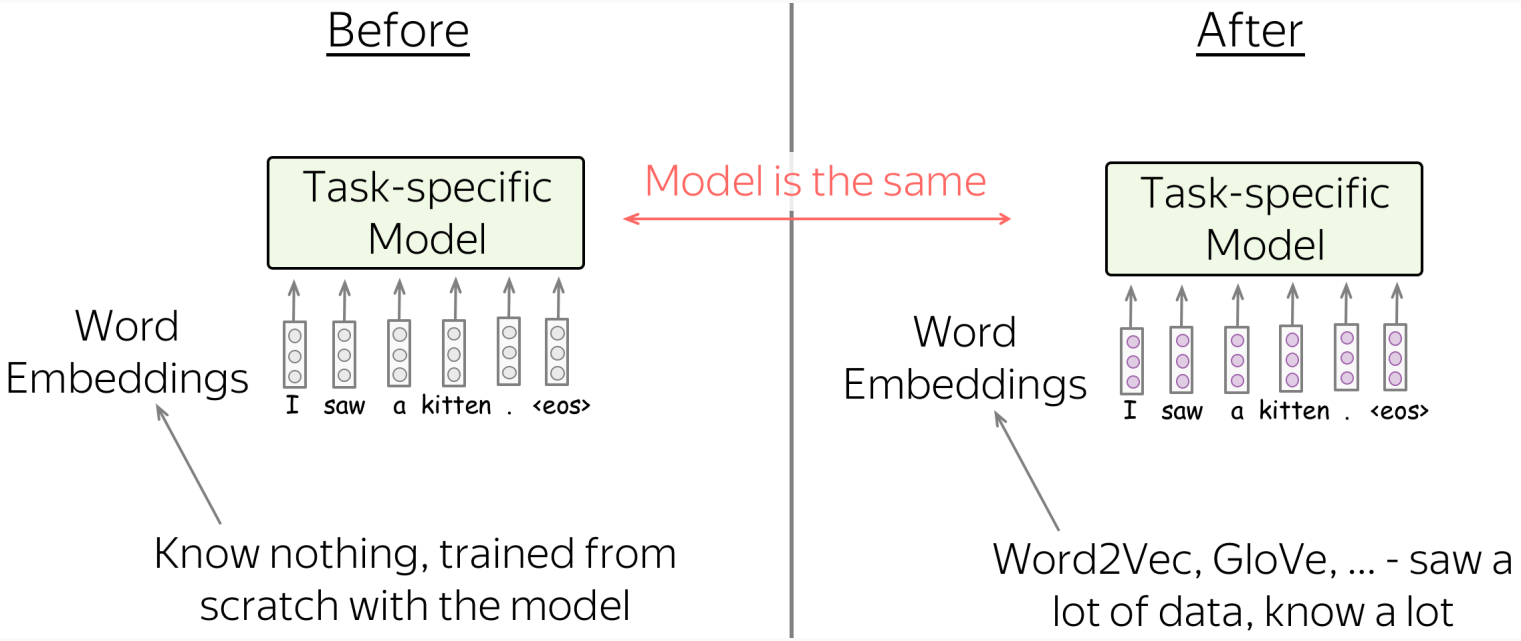
- France, Austria, Belgium, Germany, Italy, Greece, Sweden,...
- XBOX, Amiga, Playstation, MSX, iPod, SEGA, Dreamcast, ...

And various semantic relations

- apple is to apples as car is to cars
- Rome is to Italy as Paris is to France
- man is to woman as king is to queen

See Mikolov et al. [4] and Mikolov et al. [5], **but also** Bolukbasi et al. [2] which exposes systematic biases.

How to use word embeddings



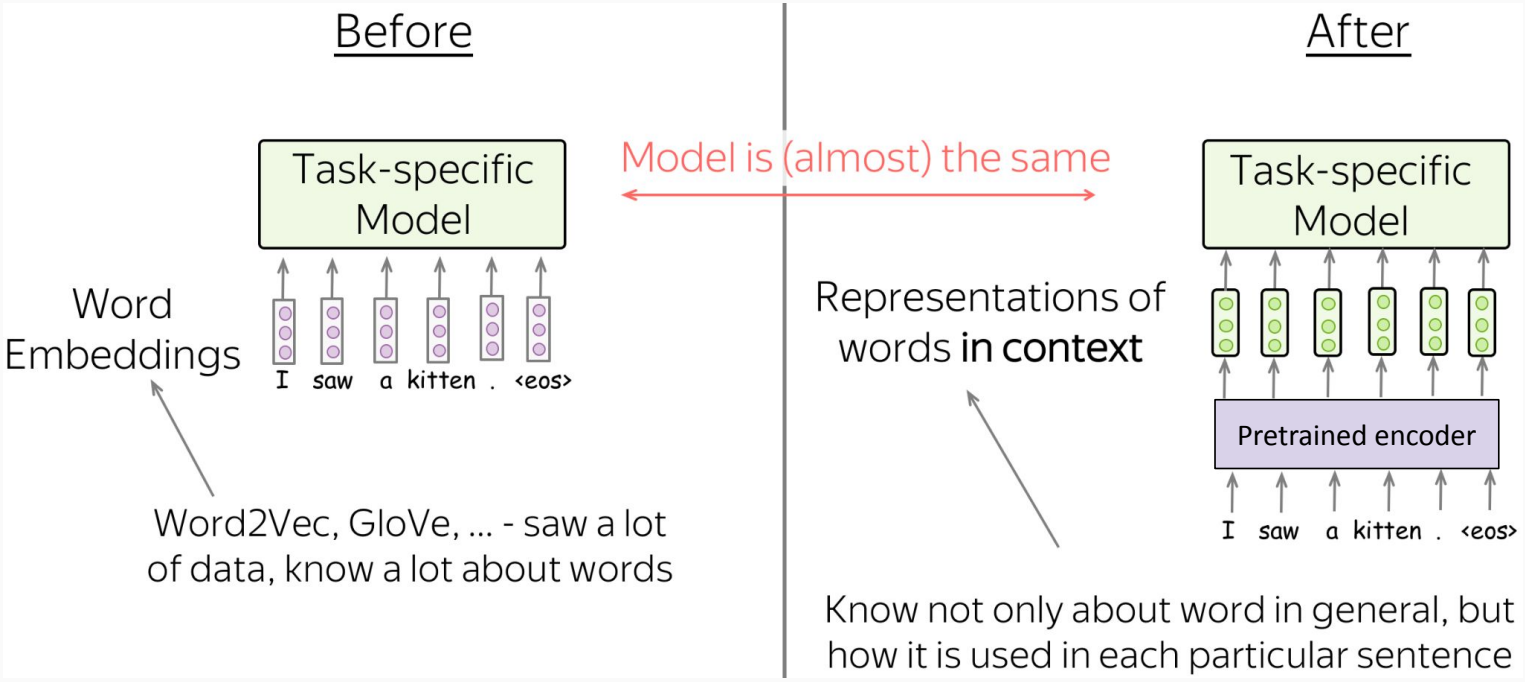
In T3, for example, you started from one such pretrained embedding layer (GloVe), but continued with untrained architecture blocks (e.g., bidirectional encoders), which you then trained from end-to-end on task data.

Use the pretrained embedding layer in combination with whatever else you need. You may 'freeze' the embeddings or 'fine-tune' them along with the rest of the model.

Beyond word representation

Skip-gram is great but it isn't an entire encoder, it's at best a way to learn a word embedding function.

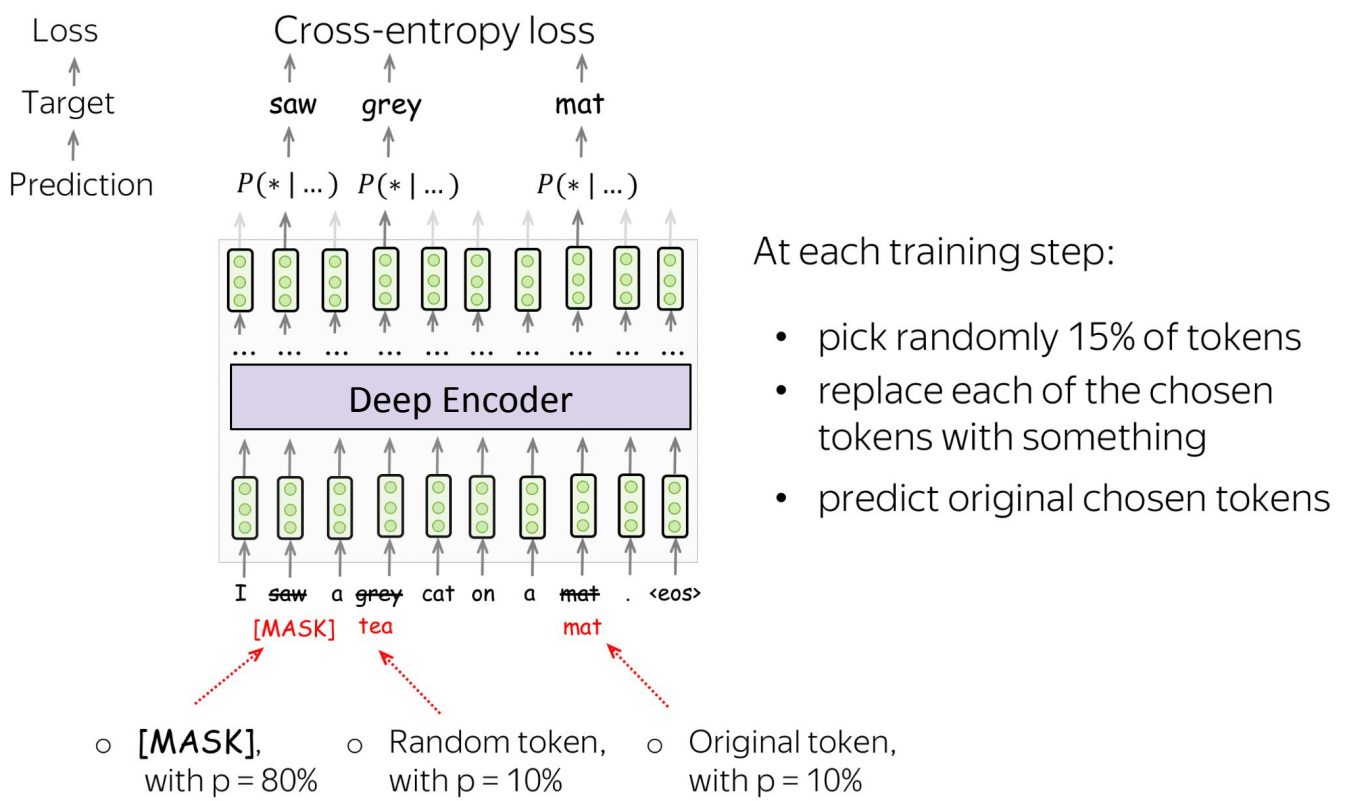
We want somethign more like this



We will now follow on this path of constructing artificial, but useful, tasks and pretrain entire encoders.

Masked language modelling (MLM)

Train a model to predict parts of the input that are corrupted at random (this is a kind of *denoising auto-encoder*).



At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

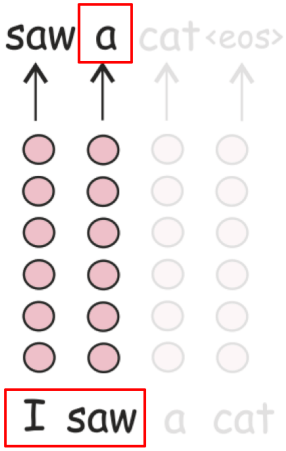
We can use any deep encoder we like. On top of it, we need only an output layer to project encodings to distributions over the vocabulary. This shallow 'read out' (or 'decoder') layer is discarded after training.

MLM vs. language modelling

The two tasks are not the same, despite the similar names. In language modelling, we generate the entire input text, incrementally from left to right. In MLM, we reconstruct parts of the text that have been corrupted at random while conditioning on the complete but corrupted version of the text.

<u>Language Modeling</u>

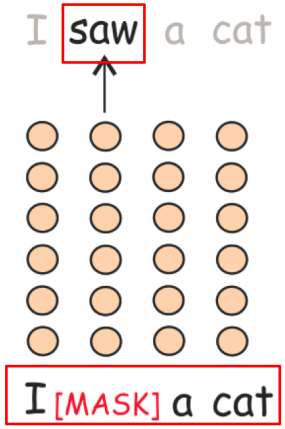
- Target: next token
- Prediction: $P(* \mid \texttt{I saw})$



left-to-right, does not see future

<u>Masked Language Modeling</u>

- Target: current token (the true one)
- Prediction: $P(* \mid \texttt{I [MASK] a cat})$



sees the whole text, but something is corrupted

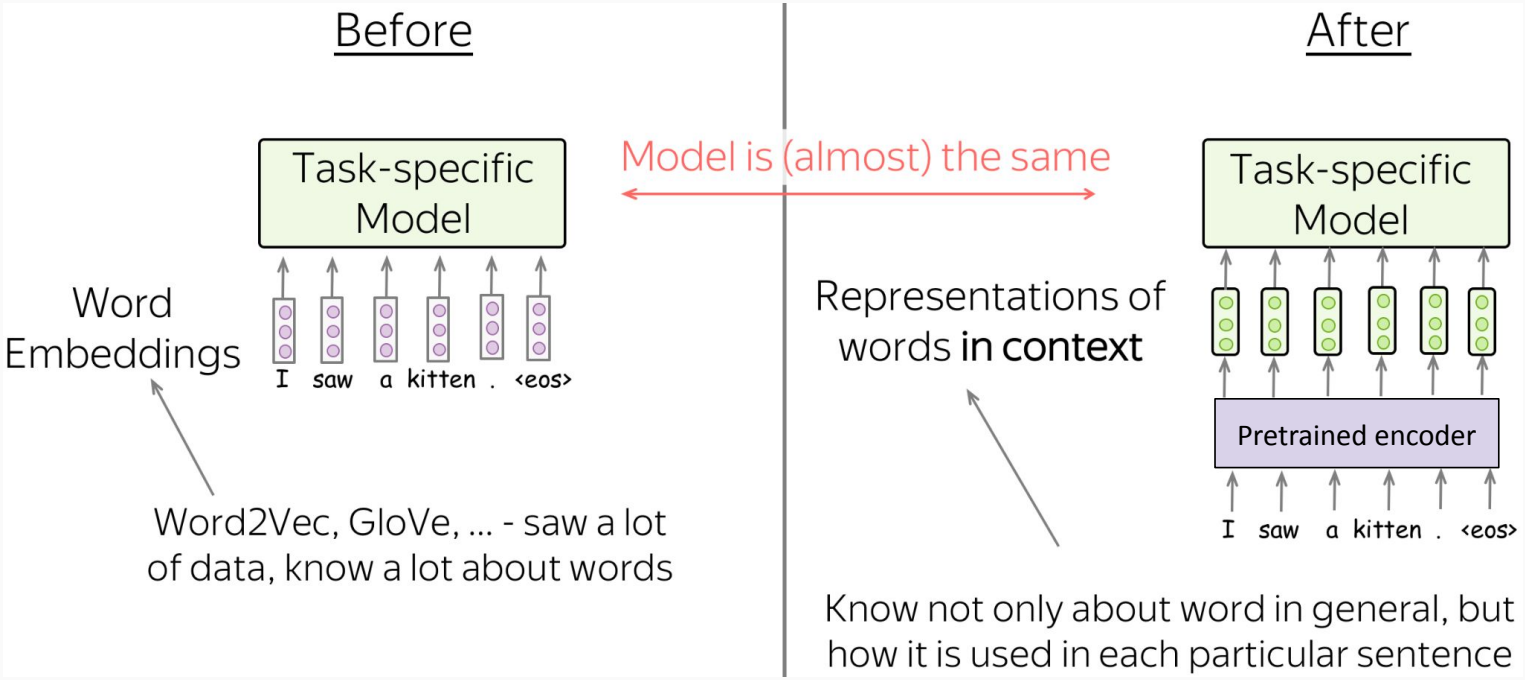
What do MLM-trained encoders learn?

Tenney et al. [6]

- Traditional linguistic hierarchy emerges within layers of BERT
- lower layers - syntax; higher layers - semantics and discourse.

How to use a pretrained encoder?

In general, just like we did with word-embeddings, we attach whatever task-specific components we need at the top:



We then fix the encoder's parameters and train the rest of the architecture on task-specific data. Or, we continue to adapt or *fine-tune* the encoder's parameters.

BERT-style encoders

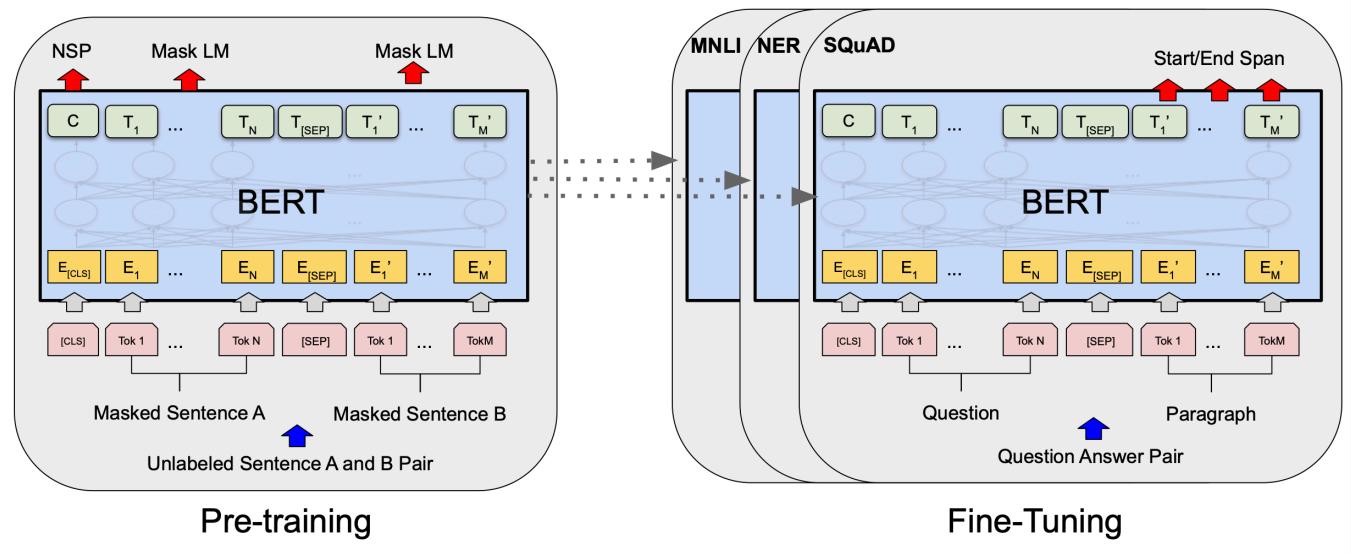
Scaling to large data

Objectives like MLM are often referred to as *self-supervision* (a bit of a misleading, but catchy term). We can create a lot of supervision of this kind, since it's derived from whatever text collections we can get our hands on.

In trying to use the largest data sources possible, researchers bumped into some problems with recurrent architectures (the state-of-the-art then for contextualised encodings): lack of parallelism, struggle with keeping and accessing long-distance dependencies, etc. These motivated the development of the Transformer architecture [7].

Bidirectional Encoder Representations from Transformers

The BERT encoder [3] is a deep Transformer stack trained via MLM (as well as 'next sentence prediction') on Books Corpus (800m words) and English Wikipedia (2500m words).



BERT is pretrained on a combination of two objectives (MLM and NSP) on a large English corpus. After training, it can be adapted to a variety of tasks via, for example, fine-tuning the architecture and whatever necessary output layers on task-specific data and loss.

Adapting a BERT-type model for classification

It's common to adapt BERT-style encoders by at most adding some 'classification heads' at the top of the architecture and fine-tuning the entire design (or keeping the encoder fixed).

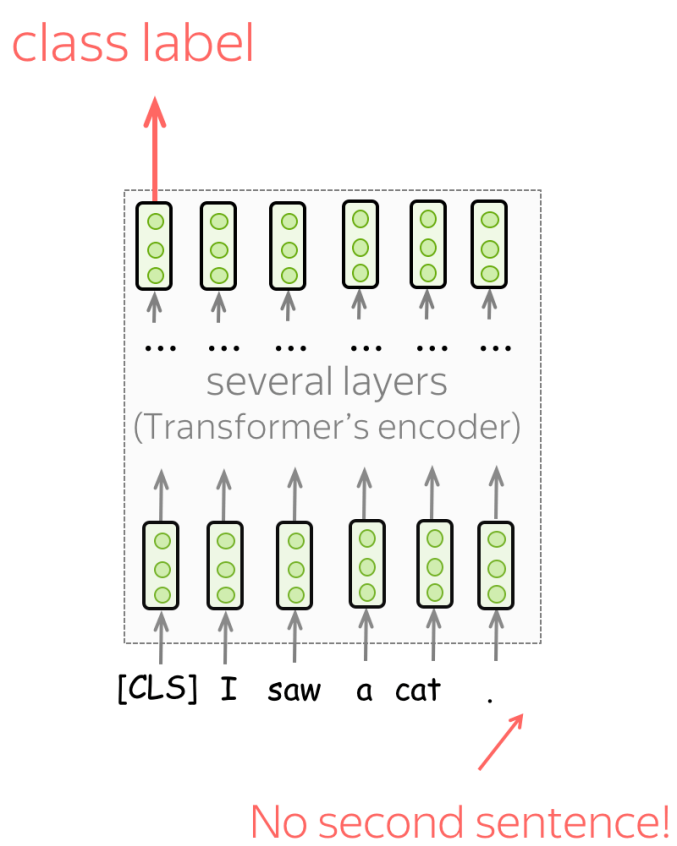
This classification head maps some conveniently chosen hidden-state at the top of the Transformer stack to the probabilities of the possible output labels. We initialise it at random and train it on task-specific data.

For K labels, the options are:

- linear ($D \to K$) and softmax
- FFNN ($D \to H \to \ldots \to K$) and softmax

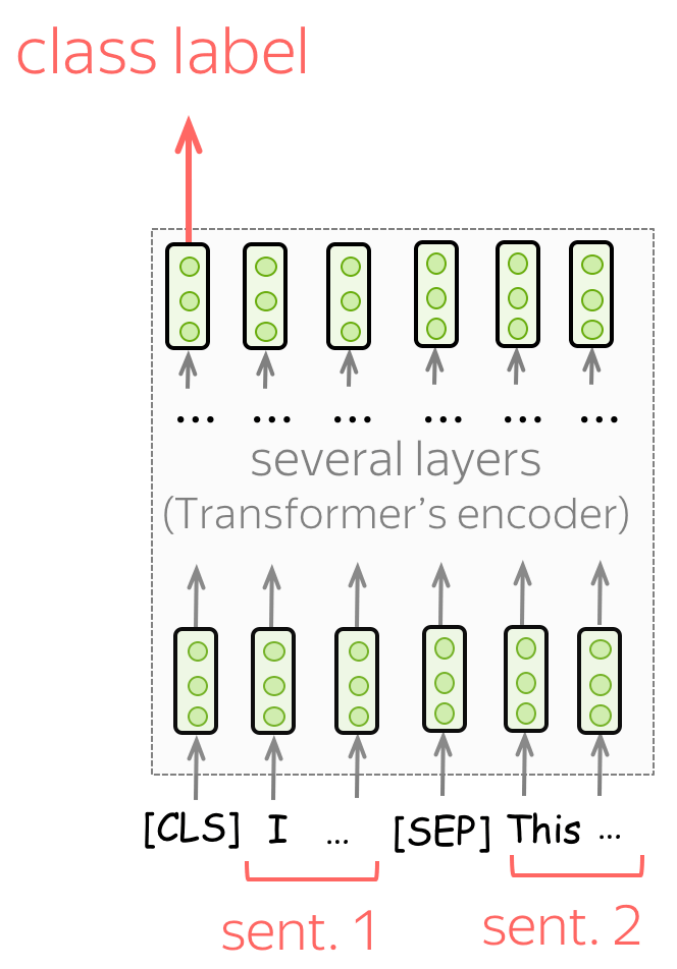
BERT-style encoders have some special tokens intended to help adapt the models to various ends.

Basic text classification



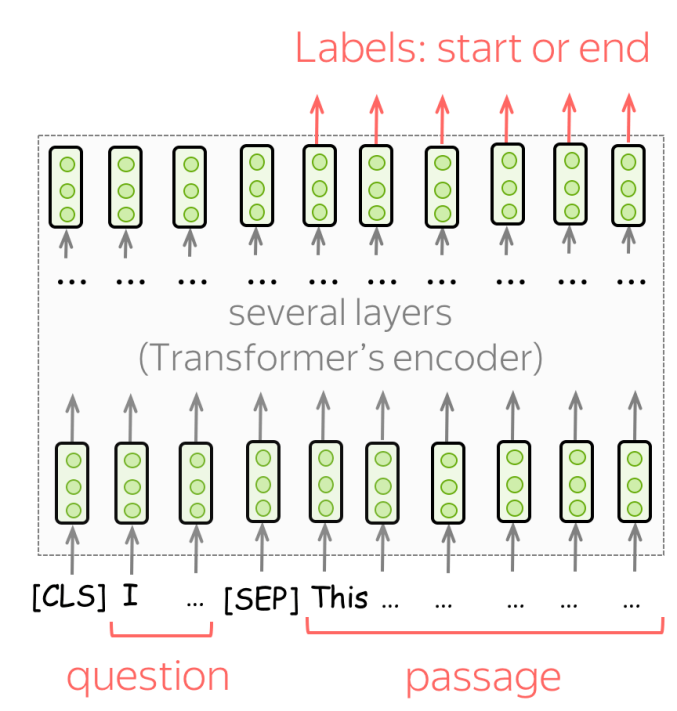
BERT was trained with a so-called [CLS] token at the beginning of sentences, we can use the corresponding hidden state at the top of the stack as an encoding of the entire text.

Text pair classification



BERT also has a special [SEP] token intended to mark the boundary between two texts. This can be used, for example, for tasks that classify multiple sentences (e.g., natural language inference). The [CLS] state can be then be used as input to the classification head.

Multiple classification heads



We can introduce multiple output classification heads. For example, in question-answering (QA) we have a head to classify a token-position as the beginning of the answer, and another to classify a token-position as the end of the answer.

And beyond

We can also find ways to adapt BERT-style encoders into regressors, taggers, and much more.

But to approach ‘generation tasks’, we need to introduce a decoder and the means to pretrain it (HC6b).

Wrap-up

BERT's impact

In NLP,

- advanced the state-of-the-art in most NLP tasks
- reduced the need for task-specific architectures
- learns representations that are predictive of various levels of linguistic representation [6]

BERT and other BERT-style encoders consolidated the **pretrain then fine-tune** programme in NLP at first and then throughout AI.

It shows that high-scale self-supervised pretraining leads to models that can be easily adapted to all sorts of applications with little design modification and requiring less additional task-specific data than previous approaches did.

Limitations

Architecture

- encoder-only models have ‘classification head’ style decoders and hence are not appropriate for generation tasks (translation, summarisation, captioning, free-form question answering, etc.)

Limitations

Architecture

- encoder-only models have ‘classification head’ style decoders and hence are not appropriate for generation tasks (translation, summarisation, captioning, free-form question answering, etc.)

Data

- these models are trained on poorly curated data and are known to encode and reproduce bias, toxicity, etc.;
- large scale scraped datasets are bound to violate copy-right laws, people’s privacy, etc.;

Limitations

Architecture

- encoder-only models have ‘classification head’ style decoders and hence are not appropriate for generation tasks (translation, summarisation, captioning, free-form question answering, etc.)

Data

- these models are trained on poorly curated data and are known to encode and reproduce bias, toxicity, etc.;
- large scale scraped datasets are bound to violate copy-right laws, people’s privacy, etc.;

Broader implications

- environmental concerns;
- as models appear to be better suited for NLP tasks across the board, they start to be prematurely used for complex tasks which they were not designed for, and they feed the creativity of developers of questionable applications;

What next?

Beyond encoders

- We can *also* pretrain decoder-only (LMs) and even encoder-decoder (conditional LMs) models, we need only find (or define) relevant artificial tasks.
- Typically, these require *even more* data than the 'encoder-only' kind of models we just discussed do. We will have an overview of those in the next class.

For applications, don't miss T6 and E6.

For improvements, more applications, and even broader implications, don't miss HC6b.

References

[1] Marco Baroni, Georgiana Dinu, and Germán Kruszewski. Don't count, predict! a systematic comparison of context-counting vs. context-predicting semantic vectors. In Kristina Toutanova and Hua Wu, editors, *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 238–247, Baltimore, Maryland, June 2014. Association for Computational Linguistics. doi: 10.3115/v1/P14-1023. URL https://aclanthology.org/P14-1023/.

[2] Tolga Bolukbasi, Kai-Wei Chang, James Y Zou, Venkatesh Saligrama, and Adam T Kalai. Man is to computer programmer as woman is to homemaker? debiasing word embeddings. *Advances in neural information processing systems*, 29, 2016.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://aclanthology.org/N19-1423/.

[4] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[5] Tomas Mikolov, Wen-tau Yih, and Geoffrey Zweig. Linguistic regularities in continuous space word representations. In Lucy Vanderwende, Hal Daumé III, and Katrin Kirchhoff, editors, *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 746–751, Atlanta, Georgia, June 2013. Association for Computational Linguistics. URL https://aclanthology.org/N13-1090/.

[6] Ian Tenney, Dipanjan Das, and Ellie Pavlick. BERT rediscovers the classical NLP pipeline. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1452. URL https://aclanthology.org/P19-1452/.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, pages 6000–6010, 2017. URL *http://papers.nips.cc/paper/7181-attention-is-all-you-need*.